

1. Java Lambda Expressions

```
import java.io.*;

import java.util.*;

interface PerformOperation {

    boolean check(int a);

}

class MyMath {

    public static boolean checker(PerformOperation p, int num) {

        return p.check(num);

    }

    PerformOperation isOdd() {

        return a -> a % 2 != 0;

    }

    PerformOperation isPrime() {

        return a -> {

            if (a < 2)

                return false;

            for (int i = 2; i <= Math.sqrt(a); i++) {

                if (a % i == 0)

                    return false;

            }

            return true;

        };

    }

    PerformOperation isPalindrome() {

        return a -> {
```

```

    int original = a;

    int reverse = 0;

    int temp = a;

    while (temp > 0) {
        reverse = reverse * 10 + temp % 10;

        temp /= 10;
    }

    return original == reverse;

};

}

}

```

```

public class Solution {

    public static void main(String[] args) throws IOException {
        MyMath ob = new MyMath();

        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        int T = Integer.parseInt(br.readLine());

        PerformOperation op;

        boolean ret = false;

        String ans = null;

        while (T--> 0) {

            String s = br.readLine().trim();

            StringTokenizer st = new StringTokenizer(s);

            int ch = Integer.parseInt(st.nextToken());

```

```
int num = Integer.parseInt(st.nextToken());

if (ch == 1) {
    op = ob.isOdd();
    ret = ob.checker(op, num);
    ans = (ret) ? "ODD" : "EVEN";
} else if (ch == 2) {
    op = ob.isPrime();
    ret = ob.checker(op, num);
    ans = (ret) ? "PRIME" : "COMPOSITE";
} else if (ch == 3) {
    op = ob.isPalindrome();
    ret = ob.checker(op, num);
    ans = (ret) ? "PALINDROME" : "NOT PALINDROME";

}

System.out.println(ans);

}

}

}
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Your Output (stdout)

```
1 EVEN
2 PRIME
3 PALINDROME
4 ODD
5 COMPOSITE
```

Expected Output

```
1 EVEN
2 PRIME
3 PALINDROME
4 ODD
5 COMPOSITE
```

[Download](#)

2. Java Comparator

```
import java.util.*;

// Write your Checker class here

class Checker implements Comparator<Player> {

    @Override
    public int compare(Player a, Player b) {
        if (a.score != b.score) {
            return b.score - a.score; // Sort by score in descending order
        }
        return a.name.compareTo(b.name); // Sort by name in ascending order
    }
}

class Player{
    String name;
    int score;

    Player(String name, int score){
        this.name = name;
        this.score = score;
    }
}

class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
```

```

int n = scan.nextInt();

Player[] player = new Player[n];

Checker checker = new Checker();

for(int i = 0; i < n; i++){
    player[i] = new Player(scan.next(), scan.nextInt());
}

scan.close();

Arrays.sort(player, checker);

for(int i = 0; i < player.length; i++){
    System.out.printf("%s %s\n", player[i].name, player[i].score);
}
}
}

```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```

1 5
2 amy 100
3 david 100
4 heraldo 50
5 aakashsha 75
6 aleksa 150

```

Your Output (stdout)

```

1 aleksa 150
2 amy 100
3 david 100
4 aakashsha 75
5 heraldo 50

```

[Download](#)

3. Java Sort

```
import java.util.*;
```

```
class Student{  
    private int id;  
    private String fname;  
    private double cgpa;  
    public Student(int id, String fname, double cgpa) {  
        super();  
        this.id = id;  
        this.fname = fname;  
        this.cgpa = cgpa;  
    }  
    public int getId() {  
        return id;  
    }  
    public String getFname() {  
        return fname;  
    }  
    public double getCgpa() {  
        return cgpa;  
    }  
}
```

```
//Complete the code
```

```
public class Solution  
{
```

```

public static void main(String[] args){

    Scanner in = new Scanner(System.in);

    int testCases = Integer.parseInt(in.nextLine());


    List<Student> studentList = new ArrayList<Student>();

    while(testCases>0){

        int id = in.nextInt();

        String fname = in.next();

        double cgpa = in.nextDouble();


        Student st = new Student(id, fname, cgpa);

        studentList.add(st);


        testCases--;

    }


    Collections.sort(studentList, new Comparator<Student>() {

@Override

public int compare(Student s1, Student s2) {

    if (Double.compare(s2.getCgpa(), s1.getCgpa()) != 0) {

        return Double.compare(s2.getCgpa(), s1.getCgpa()); // CGPA descending

    } else if (!s1.getFname().equals(s2.getFname())) {

        return s1.getFname().compareTo(s2.getFname()); // Name ascending

    } else {

        return Integer.compare(s1.getId(), s2.getId()); // ID ascending

    }

}

});

```

```
for(Student st: studentList){  
    System.out.println(st.getFname());  
}  
}
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

1	5
2	33 Rumpa 3.68
3	85 Ashis 3.85
4	56 Samiha 3.75
5	19 Samara 3.75
6	22 Fahim 3.76

Your Output (stdout)

1	Ashis
2	Fahim
3	Samara
4	Samiha
5	Rumpa

4.Sorting: Comparator

```
import java.util.*;

class Player {

    String name;

    int score;

    Player(String name, int score) {

        this.name = name;

        this.score = score;

    }

}

class Checker implements Comparator<Player> {

    // complete this method

    @Override

    public int compare(Player a, Player b) {

        if (a.score != b.score) {

            return b.score - a.score;

        }

        return a.name.compareTo(b.name);

    }

}

public class Solution {
```

```

public static void main(String[] args) {

    Scanner scan = new Scanner(System.in);

    int n = scan.nextInt();


    Player[] player = new Player[n];

    Checker checker = new Checker();


    for(int i = 0; i < n; i++){

        player[i] = new Player(scan.next(), scan.nextInt());

    }

    scan.close();


    Arrays.sort(player, checker);

    for(int i = 0; i < player.length; i++){

        System.out.printf("%s %s\n", player[i].name, player[i].score);

    }

}
}

```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

✔ Sample Test case 1

✔ Sample Test case 2

```

1  5
2  amy 100
3  david 100
4  heraldo 50
5  aakansha 75
6  aleksa 150

```

Your Output (stdout)

```

1  aleksa 150
2  amy 100
3  david 100
4  aakansha 75
5  heraldo 50

```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

✓ Sample Test case 2

Input (stdin)

```
1 3
2 smith 20
3 jones 15
4 jones 20
```

Your Output (stdout)

```
1 jones 20
2 smith 20
3 jones 15
```

Expected Output

```
1 jones 20
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

✓ Sample Test case 2

Input (stdin)

```
1 4
2 davis 15
3 davis 20
4 davis 10
5 edgehill 15
```

Your Output (stdout)

```
1 davis 20
2 davis 15
3 edgehill 15
4 davis 10
```

5. Running Sum of 1D Array

```
class Solution {  
    public int[] runningSum(int[] nums) {  
        for (int i = 1; i < nums.length; i++) {  
            nums[i] += nums[i - 1];  
        }  
        return nums;  
    }  
}
```

Testcase > Test Result

Case 1 Case 2 Case 3

Input

nums =
[1, 2, 3, 4]

Output

[1, 3, 6, 10]

Expected

[1, 3, 6, 10]

6. Richest Customer Wealth

```
class Solution {  
    public int maximumWealth(int[][] accounts) {  
        int maxWealth = 0;  
        // Iterate through each customer  
        for (int[] customer : accounts) {  
            int currentWealth = 0;  
            // Sum up money in all banks for the current customer  
            for (int bank : customer) {  
                currentWealth += bank;  
            }  
            // Update maxWealth if current customer is richer  
            maxWealth = Math.max(maxWealth, currentWealth);  
        }  
        return maxWealth;  
    }  
}
```

✓ Testcase | >_ Test Result

✓ Case 1

✓ Case 2

✓ Case 3

Input

accounts =
[[1,5], [7,3], [3,5]]

Output

10

Expected

10

7. Squares of a Sorted Array

```
class Solution {  
    public int[] sortedSquares(int[] nums) {  
        int n = nums.length;  
        int[] result = new int[n];  
        int left = 0;  
        int right = n - 1;  
        int index = n - 1; // Fill the result array from back to front  
        while (left <= right) {  
            int leftSquare = nums[left] * nums[left];  
            int rightSquare = nums[right] * nums[right];  
  
            if (leftSquare > rightSquare) {  
                result[index] = leftSquare;  
                left++;  
            } else {  
                result[index] = rightSquare;  
                right--;  
            }  
            index--;  
        }  
  
        return result;  
    }  
}
```

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

Input

```
nums =  
[-4,-1,0,3,10]
```

Output

```
[0,1,9,16,100]
```

Expected

```
[0,1,9,16,100]
```

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

Input

```
nums =  
[-7,-3,2,3,11]
```

Output

```
[4,9,9,49,121]
```

Expected

```
[4,9,9,49,121]
```

2. Find Pivot Index

```
class Solution {  
    public int pivotIndex(int[] nums) {  
        int totalSum = 0;  
        int leftSum = 0;  
  
        // Step 1: Calculate the total sum of the array  
        for (int num : nums) {  
            totalSum += num;  
        }  
  
        // Step 2: Iterate and check the pivot condition  
        for (int i = 0; i < nums.length; i++) {  
            // Right sum is totalSum - leftSum - nums[i]  
            if (leftSum == totalSum - leftSum - nums[i]) {  
                return i; // Return the leftmost index immediately  
            }  
            leftSum += nums[i];  
        }  
  
        return -1;  
    }  
}
```

✓ Testcase | >_ Test Result

Accepted Runtime: 0 ms

✓ Case 1 ✓ Case 2 ✓ Case 3

Input

```
nums =  
[1,7,3,6,5,6]
```

Output

3

Expected

3

✓ Testcase | >_ Test Result

✓ Case 1 ✓ Case 2 ✓ Case 3

Input

```
nums =  
[1,2,3]
```

Output

-1

Expected

-1

3 cases